

POSTECH



TECH CHALLENGE

Tech Challenge é o projeto da fase que englobará os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, a princípio, deve ser desenvolvida em grupo. É importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale 90% da nota de todas as disciplinas da fase.

Atividade em grupo · Obrigatória · Avaliada.

Entrega obrigatória: Repositório GitHub + Vídeo de 5 minutos (método STAR).

Entrega opcional: Deploy em ambiente de produção em nuvem (AWS, Azure ou GCP).

O problema

Uma empresa de e-commerce precisa de um sistema de recomendação de produtos baseado no comportamento de navegação dos usuários. O modelo central é uma rede neural (MLP ou embedding-based) treinada com PyTorch, com pipeline completo containerizado em Docker, dados versionados com DVC, experimentos rastreados no MLflow e código seguindo padrões profissionais de clean code.

Requisitos Obrigatórios

Repositório github

- Estrutura clean code: módulos curtos, nomes descritivos, SOLID, type hints.
- pyproject.toml com Poetry/uv, dependências prod/dev separadas, lock file commitado.
- .dockerignore, .gitignore, .env.example configurados.
- Histórico de commits semântico.

Vídeo (5 minutos — método STAR)

- **Situation:** Problema de negócio e contexto do dataset.
- **Task:** Objetivos técnicos e restrições.
- **Action:** Decisões de arquitetura, modelo, versionamento e containerização.
- **Result:** Resultados obtidos, trade-offs e lições aprendidas.

Bibliotecas Requeridas

- **PyTorch** — rede neural para o modelo de recomendação.
- **Scikit-Learn** — pré-processamento e baselines.
- **MLflow** — tracking de experimentos e Model Registry.
- **DVC** — versionamento de dados e pipeline reprodutível.

Boas Práticas Obrigatórias

- Clean code: funções ≤ 20 linhas, naming conventions, type hints.
- Design patterns aplicados (Factory, Strategy ou Template Method).
- Dockerfile multi-stage com imagem otimizada.
- Pipeline DVC com ≥ 3 stages.
- Seeds fixados, lock file, .env.

Etapas de Desenvolvimento (4 Etapas)**Etapas 1 — Clean Code e Estrutura (Disciplina 01)**

Foco: Projeto limpo com padrões de engenharia desde o início.

Tarefa	Referência
Definir estrutura de projeto com src/, tests/, data/, models/, configs/	Clean Code, Aula 01
Aplicar naming conventions e SOLID desde a primeira linha.	Clean Code, Aulas 01–02
Implementar ≥ 1 design pattern (Factory para criar modelos, Strategy para preprocessors).	Clean Code, Aula 03
Type hints em todas as funções públicas + docstrings Google style.	Clean Code, Aula 03
Configurar ruff sem erros + pre-commit hooks.	Clean Code, Aula 03

Entregável: Repositório base com estrutura limpa e linting passando.

Etapa 2 — Ambiente e Dependências (Disciplina 02)

Foco: Reprodutibilidade garantida com gerenciamento moderno de dependências.

Tarefa	Referência
Configurar pyproject.toml com Poetry: deps de prod (pytorch, sklearn, mlflow) e dev (pytest, ruff).	Dependências, Aula 02
Gerar e commitar lock file.	Dependências, Aula 03
Externalizar configurações para .env + Pydantic Settings.	Dependências, Aula 03
Script de validação de ambiente (scripts/validate_env.py).	Dependências, Aula 01

Verificar instalação limpa em ambiente novo.	Dependências, Aula 02
--	-----------------------

Entregável: Projeto instalável do zero com `poetry install`.

Etapa 3 — Containerização e Versionamento (Disciplinas 03 e 04)

Foco: Docker + DVC + MLflow integrados em pipeline reprodutível.

Tarefa	Referência
Dockerfile multi-stage: builder (deps) + runtime (app).	Docker, Aulas 01–03
docker-compose.yml com serviço de treino + MLflow server.	Docker, Aula 04
DVC init, versionar dataset, configurar remote (local ou S3).	DVC+MLflow, Aulas 01–03
Pipeline DVC (dvc.yaml): preprocess → feature_eng → train → evaluate.	DVC+MLflow, Aula 04
MLflow tracking: logar params, métricas, artefatos de cada run.	DVC+MLflow, Aula 04

Entregável: Pipeline reprodutível via `dvc repro` + Docker funcional.

Etapa 4 — Rede Neural, Registry e Entrega (Disciplina 04 + consolidação)

Foco: Modelo neural treinado, registrado e documentado.

Tarefa	Referência
Treinar MLP/embedding model com PyTorch para recomendação.	— (PyTorch)
Comparar com baselines (Scikit-Learn) usando ≥ 4 métricas.	— (Scikit-Learn)

Registrar modelo no MLflow Model Registry → Staging → Production.	DVC+MLflow, Aula 05
Escrever Model Card com performance, limitações e vieses.	— (boas práticas)
Finalizar README com instruções completas.	—
Gravar vídeo STAR de 5 minutos.	—
(Opcional) Deploy em nuvem via Docker.	—

Entregável: Repositório final + modelo no Registry + vídeo STAR.

Critérios de Avaliação

Critério	Peso	Descrição
Clean code e estrutura.	15%	SOLID, naming, type hints, design patterns, linting.
Reprodutibilidade.	15%	Poetry, lock file, .env, instalação limpa.
Docker.	15%	Multi-stage, imagem otimizada, compose funcional.
DVC + Pipeline.	15%	Dataset versionado, pipeline ≥ 3 stages, dvc repro funcional.
Rede neural (PyTorch).	15%	MLP funcional, early stopping, comparação com baselines.
MLflow + Registry.	10%	≥ 3 runs rastreados, modelo promovido a Production.
Vídeo STAR.	10%	Clareza, cobertura dos 4 elementos, ≤ 5 min.

Bônus: deploy em nuvem.	5%	Container acessível via URL pública.
-------------------------	----	--------------------------------------

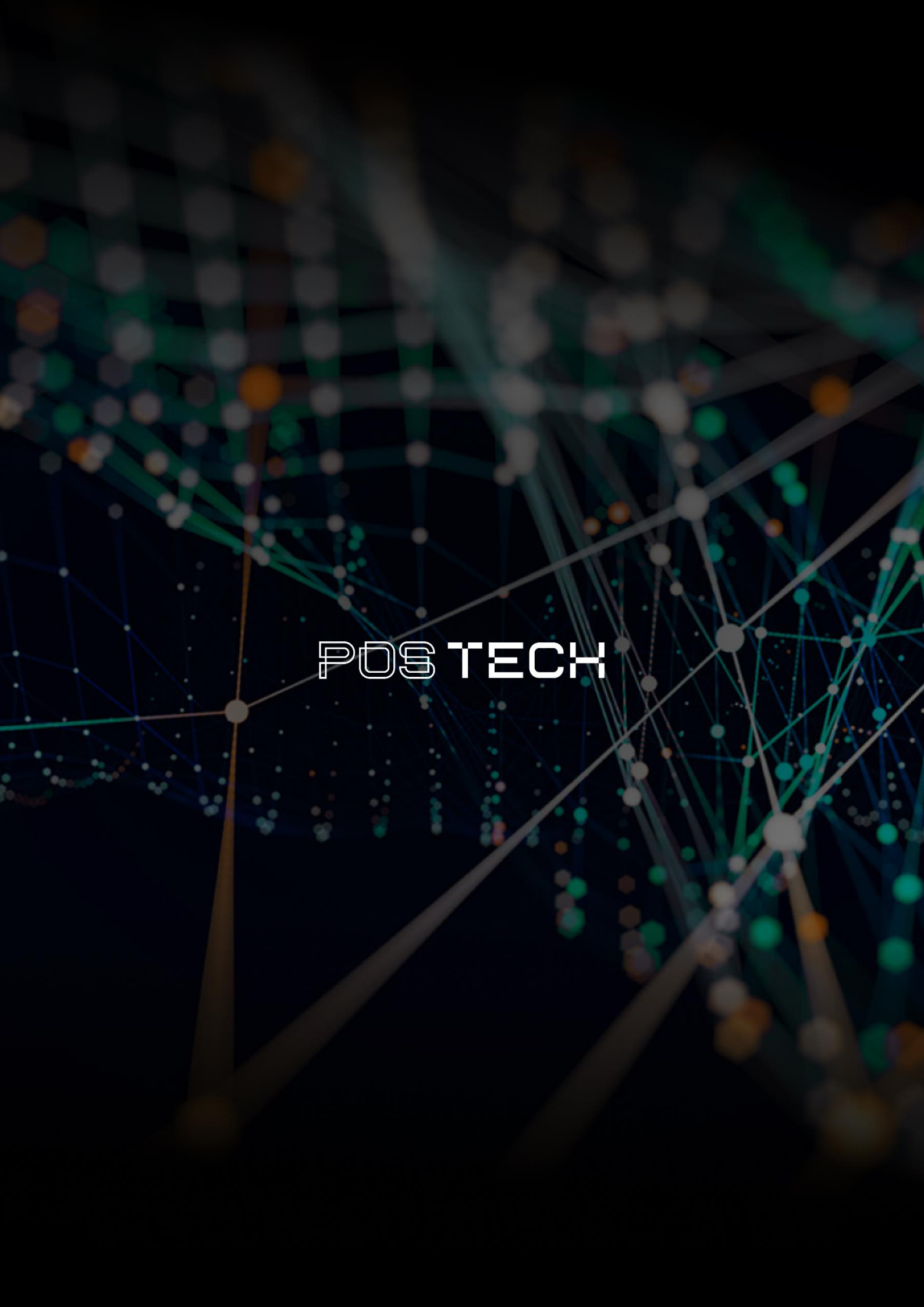
Dataset Sugerido

Dataset de interações de e-commerce (ex.: Instacart Market Basket, RetailRocket, ou MovieLens para recomendação). Alternativa: qualquer dataset com ≥ 10.000 interações user-item.

Passo a Passo Resumido

- [Etapa 1] Estrutura clean code + design patterns + linting.
- [Etapa 2] Poetry + lock file + .env + validação de ambiente.
- [Etapa 3] Docker multi-stage + DVC pipeline + MLflow tracking.
- [Etapa 4] MLP PyTorch + Model Registry + Model Card + vídeo STAR.

Caso tenha qualquer dúvida, não deixe de nos procurar no Discord!

The background is a dark, abstract network visualization. It features a complex web of thin, light-colored lines connecting numerous small, glowing nodes. The nodes are primarily white and light blue, with some larger, more prominent nodes in teal and orange. The overall effect is a sense of dynamic connectivity and data flow, typical of a digital or technological theme.

POSTECH